Course Outline: Separation by Domains and Responsibilities in Python

Title: Separation by Domains and Responsibilities in Python **Learnpack:** + Separación por dominios o responsabilidades en Python **Prerequisite:** Python fundamentals (functions, lists, dictionaries) **Target Audience:** Beginner developers learning backend organization **Total Lessons:** 12 **Estimated Total Length:** ~6,200 words **Format:** Conceptual (0% hands-on)

Course Description

As Python projects grow beyond a single file, developers face a critical challenge: how to organize code so it remains readable, maintainable, and free of frustrating errors. This course teaches students the foundational skills for structuring Python projects professionally.

Students will learn PEP8 — Python's official style guide — and understand why consistent code style matters in collaborative environments. They'll master the module and package system, learning how to split code across files and folders while maintaining clean import relationships. The course addresses one of the most common beginner frustrations: circular imports, teaching students to recognize and prevent them before they occur.

By the end of this course, students will be able to read and write Python code that follows industry standards, organize multi-file projects with confidence, and understand the import system well enough to debug common issues independently.

Course Philosophy

This course emphasizes:

- **Reading before writing:** Students learn to recognize good structure in existing code before creating their own
- **Prevention over debugging:** Understanding why problems occur prevents hours of frustration
- **Path of least resistance:** Simple, practical rules that work in 90% of cases

---

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome	1	~450
01 - PEP8	3	~1,500
02 - Modules and Packages	3	~1,550
03 - Import Mastery	2	~1,100
04 - Assessment	2	~1,150
05 - Conclusion	1	~450
Total	**12**	**~6,200**

Section 00: Welcome

00.0 Why Code Organization Matters 📖 (~450 words)

Learning Objectives:

- Recognize the problems that arise from disorganized code
- Understand how code organization impacts collaboration and maintenance
- Preview the key concepts covered in this course

Content Outline:

- The single-file trap: what happens when your project outgrows one file
- Real consequences of messy code: harder debugging, team conflicts, wasted time
- The three pillars of Python organization: style (PEP8), structure (modules), and imports
- What you'll be able to do after this course

Transitions: This lesson sets the stage by showing students the "why" before diving into the "how." It connects their previous experience writing functions to the next level of organization.

Key Principles:

- Code is read more often than it is written
- Organization is a skill that compounds over time
- Professional developers follow conventions, not personal preferences

Examples:

- A 500-line single file vs. the same logic split across 5 organized modules
- A team scenario where inconsistent style causes merge conflicts

---

Section 01: PEP8 — The Python Style Guide

01.0 What is PEP8 and Why Should You Care? 📖 (~500 words)

Learning Objectives:

- Define what PEP8 is and its role in the Python ecosystem
- Explain why style consistency matters more than personal preference
- Identify where to find PEP8 documentation

Content Outline:

- PEP8: Python Enhancement Proposal #8, the official style guide
- Why Python has an official style guide (readability counts)
- The goal: code that looks like it was written by one person
- PEP8 is a guide, not a law — but following it makes life easier
- Tools that enforce PEP8 automatically (brief mention of linters)

Transitions: From understanding what PEP8 is, students move to learning its core rules in the next lesson.

Key Principles:

- Consistency within a project matters more than any single rule
- Style guides reduce cognitive load when reading code
- AI coding assistants generally follow PEP8 — knowing it helps you review their output

Examples:

- The Zen of Python (`import this`) and its connection to PEP8
- How open-source projects use PEP8 to onboard new contributors quickly

---

01.1 Core PEP8 Rules: Naming, Spacing, and Structure 📖 (~550 words)

Learning Objectives:

- Apply correct naming conventions for variables, functions, classes, and constants
- Use proper indentation, spacing, and line length
- Structure code blocks consistently

Content Outline:

- Naming conventions:
  - `snake_case` for variables and functions
  - `PascalCase` for classes
  - `UPPER_SNAKE_CASE` for constants
- Indentation: 4 spaces, never tabs
- Line length: 79 characters (why this matters)
- Blank lines: 2 between top-level definitions, 1 between methods
- Spaces around operators and after commas
- No trailing whitespace

Transitions: Now that students know the rules, the next lesson shows them how to recognize clean vs. messy code in practice.

Key Principles:

- Naming reveals intent — `calculate_total_price` beats `calc` or `x`
- Whitespace is a tool for readability, not decoration
- These rules become automatic with practice

Examples:

- Side-by-side comparison: function with poor naming vs. clear naming
- Code block showing proper spacing around operators and arguments

---

01.2 Reading and Recognizing Clean Python Code 📖 (~450 words)

Learning Objectives:

- Identify PEP8 violations in existing code
- Distinguish between critical violations and minor style issues
- Develop the habit of scanning code for style before logic

Content Outline:

- The "code smell" of poor style: warning signs that code needs attention
- Critical violations: inconsistent naming, wrong indentation, no spacing
- Minor issues: line slightly over 79 chars, extra blank lines
- Reading code like a reviewer: style first, then logic
- How AI-generated code sometimes breaks PEP8 (and how to catch it)

Transitions: With PEP8 foundations in place, students are ready to learn how to organize code across multiple files using modules.

Key Principles:

- Train your eye to spot issues before running the code
- Fixing style issues early prevents them from multiplying
- Good style makes debugging easier because the code is predictable

Examples:

- A code snippet with 5 hidden PEP8 violations for students to identify
- Before/after comparison of the same function cleaned up

Section 02: Modules and Packages

02.0 What is a Module? Organizing Code into Files 📖 (~500 words)

Learning Objectives:

- Define what a Python module is
- Explain the relationship between files and modules
- Identify when to split code into separate modules

Content Outline:

- A module is simply a `.py` file containing Python code
- Why split code: separation of concerns, reusability, readability
- The mental model: one module = one responsibility
- Common module types: utilities, models, routes, config
- How Python finds modules (brief intro to `sys.path`)

Transitions: Understanding single modules leads to the next concept: grouping modules into packages with `__init__.py`.

Key Principles:

- A module should do one thing well
- File names become import names — choose them carefully
- Splitting too early is as problematic as splitting too late

Examples:

- A project with `main.py`, `utils.py`, and `config.py` — each with a clear purpose
- Comparison: a 300-line file vs. three 100-line modules

02.1 The `__init__.py` File and Package Structure 📖 (~550 words)

Learning Objectives:

- Define what a Python package is
- Explain the purpose of `__init__.py`
- Create a basic package structure

Content Outline:

- A package is a folder containing modules (and an `__init__.py`)
- The role of `__init__.py`: marks a folder as a Python package
- What goes inside `__init__.py`: often empty, sometimes exports
- Basic package structure example:

```
my_project/
├── main.py
└── helpers/
    ├── __init__.py
    ├── math_utils.py
    └── string_utils.py
```

- Modern Python (3.3+): implicit namespace packages (brief mention)

Transitions: With packages understood, students need to learn how to actually import from them — absolute vs. relative imports.

Key Principles:

- Start with `__init__.py` empty until you have a reason to add exports
- Package structure should mirror logical separation of concerns
- Flat is better than nested (avoid deep folder hierarchies)

Examples:

- A simple package structure for a web API: `routes/`, `models/`, `utils/`
- What happens when you forget `__init__.py` (the import error)

02.2 Absolute vs Relative Imports 📖 (~500 words)

Learning Objectives:

- Distinguish between absolute and relative imports
- Write correct syntax for both import styles
- Choose the appropriate import style for different situations

Content Outline:

- Absolute imports: full path from project root
  - `from helpers.math_utils import calculate_total`
- Relative imports: path relative to current file
  - `from .math_utils import calculate_total`
  - `from ..other_package import something`
- The dot notation: `.` = current package, `..` = parent package
- When to use which:
  - Absolute: most of the time, clearer and explicit
  - Relative: within tightly coupled packages, refactoring-friendly
- PEP8 recommendation: absolute imports are preferred

Transitions: Students now understand the two import styles. Next, they'll learn import aliasing and best practices to write clean import statements.

Key Principles:

- Explicit is better than implicit — absolute imports show the full path
- Relative imports make sense inside a package that might be renamed
- Consistency matters: pick one style per project and stick with it

Examples:

- The same import written both ways, side by side
- A package renamed from `helpers` to `utils` — relative imports still work

---

Section 03: Import Mastery

03.0 Import Aliasing and Best Practices 📖 (~550 words)

Learning Objectives:

- Use the `as` keyword to create import aliases
- Apply standard aliasing conventions (e.g., `import numpy as np`)
- Write clean, organized import sections

Content Outline:

- Import aliasing with `as`:
  - `import very_long_module_name as short`
  - `from package import ClassName as Alias`
- Why alias: shorter references, avoiding name conflicts, conventions
- Standard conventions you'll encounter:
  - `import numpy as np`
  - `import pandas as pd`
  - `from typing import List as L` (less common)
- Organizing imports (PEP8 order):
  1. Standard library imports
  2. Third-party imports
  3. Local application imports
  4. Blank line between each group
- One import per line vs. grouped imports

Transitions: With solid import skills, students need to understand the most common import problem: circular imports.

Key Principles:

- Only alias when it improves readability or follows convention
- Import order is not just style — it helps you spot missing dependencies
- Avoid `from module import *` — it pollutes the namespace

Examples:

- A well-organized import section with all three groups
- Name conflict scenario: two modules both have a `User` class

03.1 Avoiding Circular Imports 📖 (~550 words)

Learning Objectives:

- Define what a circular import is and why it causes errors
- Identify code structures that lead to circular imports
- Apply strategies to prevent and fix circular imports

Content Outline:

- What is a circular import: A imports B, B imports A
- Why it breaks: Python can't finish loading either module
- The error you'll see: `ImportError` or `AttributeError` on partially loaded module
- Common causes:
  - Two modules that depend on each other
  - Importing at the top level when you only need the import inside a function
- Prevention strategies:
  - Restructure: move shared code to a third module
  - Import inside functions (lazy import)
  - Import the module, not the attribute: `import module` vs `from module import func`
- The design smell: circular imports often signal poor separation of concerns

Transitions: Students now have all the tools for proper code organization. The next section consolidates anti-patterns and tests their understanding.

Key Principles:

- Circular imports are a symptom, not the disease — fix the structure
- If two modules need each other, they might belong together (or need a third module)
- Lazy imports are a valid escape hatch but shouldn't be the default

Examples:

- A circular import scenario: `models.py` imports from `utils.py`, `utils.py` imports from `models.py`
- The fix: extracting shared code into `common.py`

Section 04: Assessment

04.0 Common Import Mistakes (Anti-patterns) 📖 (~550 words)

Learning Objectives:

- Recognize common anti-patterns in Python code organization
- Understand the consequences of each anti-pattern
- Connect anti-patterns to the correct approaches learned earlier

Content Outline:

- Anti-pattern 1: `from module import *`
  - Problem: namespace pollution, unclear dependencies
  - Fix: explicit imports
- Anti-pattern 2: Inconsistent naming (mixing `camelCase` and `snake_case`)
  - Problem: cognitive load, looks unprofessional
  - Fix: follow PEP8 naming conventions
- Anti-pattern 3: Giant `__init__.py` with all the logic
  - Problem: defeats the purpose of packages
  - Fix: keep `__init__.py` minimal, logic in modules
- Anti-pattern 4: Deep nesting (`from a.b.c.d.e import f`)
  - Problem: fragile, hard to refactor
  - Fix: flatter structure, re-export from higher-level `__init__.py`
- Anti-pattern 5: Ignoring import order
  - Problem: hard to see dependencies at a glance
  - Fix: standard library → third-party → local

Transitions: With anti-patterns clear, students are ready to test their complete understanding in the assessment.

Key Principles:

- Anti-patterns are easy to create, harder to fix later
- Most anti-patterns stem from taking shortcuts early
- Clean organization is faster in the long run

Examples:

- Code snippet showing a messy import section with 3 anti-patterns
- The same code cleaned up following best practices

04.1 Module Organization Knowledge Check 🧠 (~600 words)

Learning Objectives:

- Demonstrate understanding of PEP8 naming conventions
- Apply correct import syntax and strategies
- Identify and explain circular import problems

Question Format: Multiple choice

Questions:

1. According to PEP8, which naming convention should be used for a Python function?

   - A) `calculateTotalPrice`
   - B) `CalculateTotalPrice`
   - C) `calculate_total_price`
   - D) `CALCULATE_TOTAL_PRICE`

2. What is the primary purpose of an `__init__.py` file in a folder?

   - A) To store the main application logic
   - B) To mark the folder as a Python package
   - C) To list all dependencies for the package
   - D) To configure the Python interpreter

3. Which import statement uses relative import syntax?

   - A) `import helpers.math_utils`
   - B) `from helpers import math_utils`
   - C) `from .helpers import math_utils`
   - D) `import math_utils as helpers`

4. A circular import occurs when:

   - A) You import the same module twice in one file
   - B) Module A imports Module B, and Module B imports Module A
   - C) You use relative imports instead of absolute imports
   - D) You forget to include `__init__.py` in a package

5. According to PEP8, what is the correct order for organizing imports?

   - A) Local → Third-party → Standard library
   - B) Alphabetical order regardless of source
   - C) Standard library → Third-party → Local application
   - D) Third-party → Standard library → Local

6. Which strategy helps prevent circular imports?

   - A) Always use relative imports
   - B) Move shared code to a third module that both can import
   - C) Use `from module import *` to get everything at once
   - D) Avoid using `__init__.py` files

7. What is the main problem with using `from module import *`?

   - A) It's slower than other import methods
   - B) It only works with standard library modules
   - C) It pollutes the namespace and hides dependencies
   - D) It causes circular imports

Evaluation Criteria:

- 6-7 correct: Excellent understanding of Python organization
- 4-5 correct: Good grasp, review specific weak areas
- Below 4: Revisit the conceptual lessons before proceeding

Section 05: Conclusion

05.0 Building Maintainable Python Projects (~450 words)

Content Outline:

Course Recap: You've learned the three pillars of Python code organization: style (PEP8), structure (modules and packages), and imports (absolute, relative, and aliases). These skills transform you from someone who writes Python that works to someone who writes Python that lasts.

What You Accomplished:

- Mastered PEP8 naming, spacing, and structure conventions
- Understood how modules and packages organize code across files
- Learned to write clean imports and avoid circular dependencies
- Identified common anti-patterns that cause maintenance headaches

Connection to the Bigger Picture: These organization skills become critical as you build backends with FastAPI. Every route, model, and utility will live in its own module. Teams you work with will expect PEP8-compliant code. AI coding assistants produce better results when your project structure is clear.

Next Steps:

- Start applying PEP8 to your existing code — even small fixes build the habit
- When your next file grows beyond 100 lines, practice splitting it into modules
- Install a linter (like `flake8` or `ruff`) to catch style issues automatically
- Review open-source Python projects to see professional structure in action

Resources for Further Exploration:

- PEP8 official documentation: https://peps.python.org/pep-0008/
- Python Packaging User Guide: https://packaging.python.org/
- Real Python's guide to Python modules and packages

Encouragement: Code organization feels like extra work at first, but it's an investment that pays dividends. Every minute spent organizing saves ten minutes debugging. Every naming convention followed is one less decision to make later. You're building habits that separate professional developers from hobbyists. Keep practicing — clean code becomes second nature.

Quality Verification

Structure ✓

- [x] 12 total lessons (optimized for conceptual content)
- [x] 6 sections including welcome and outro
- [x] Logical progression: style → structure → imports → anti-patterns
- [x] Each content section has 2-3 lessons
- [x] Anti-pattern lesson (04.0) placed AFTER teaching correct approaches
- [x] Assessment (🧠) in second-to-last section
- [x] Final section has exactly 1 lesson (outro only)
- [x] Every content lesson marked with emoji (📖 or 🧠)

Content ✓

- [x] Specific, measurable learning objectives
- [x] Concrete, relevant examples in each lesson
- [x] Logical transitions between lessons
- [x] Key principles aligned with 4Geeks philosophy
- [x] No repetition from previous learnapacks (functions, lists covered in Day 20)
- [x] No premature coverage (file I/O comes in Day 22)

Syllabus Compliance ✓

- [x] Extracted from ONE `+` block only
- [x] All bullets covered: PEP8, imports (absolute/relative), `__init__.py`, aliasing, circular imports
- [x] No content from other `+` blocks (no architecture, no FastAPI)